

51. Sauter, sans pesanteur

Compter les images qui restent à monter, plutôt qu'une vitesse signée.

1. Ce qu'il faut comprendre

Le front sert enfin : sans lui, garder `A` enfoncé relancerait le saut à chaque image, et le personnage collerait au plafond.

`montee` compte les images qui restent à monter. C'est plus simple qu'une vraie vitesse signée — et sur une machine **sans nombres négatifs**, plus simple veut dire plus juste.

La leçon « La pesanteur et le saut » reprend la même idée avec une vraie accélération.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **337 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	47 o	← la console, à l'allumage — la mise en route de la console
main()	118 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	16 o	← CopierTuiles — 0 lettre sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

sprite() × 1	23 o sur place, dans main()
bouton() × 1	12 o sur place, dans main() + LireManette (37 o)
image() × 1	6 o sur place, dans main() + AttendreImage (28 o)

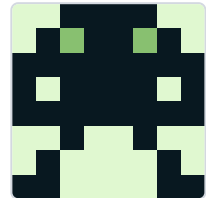
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : A fait sauter le héros ; il monte, puis redescend jusqu'au sol.
//
// « montée » compte les images de montée qui restent. Tant qu'il en reste,
// on monte de 2 pixels ; ensuite, on redescend de 2 pixels jusqu'au sol.
// (Pas de pesanteur ici : la vitesse ne change pas.)

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {
    "..###..", ".#-##-#.", "#####", "#.####.#",
    "#####", "..#..#..", ".#....#.", "##....##",
};
```



à l'écran

3. UNE CONSTANTE — SOL

```
const uint8_t SOL = 120;    // La hauteur du sol, en pixels.
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t y = SOL;        // Le héros commence au sol.
    uint8_t montee = 0;     // Les images de montée qui restent (0 = il ne monte pas).
    uint8_t avant = 0;      // A était-elle enfoncée au tour d'avant ?

    while (true) {          // Répète pour toujours :
        uint8_t appui = bouton(A); // A maintenant ?

        // On ne saute qu'au FRONT, et seulement depuis le sol.
        if (appui && !avant && y == SOL) montee = 16; // 16 images de montée
        avant = appui;

        if (montee > 0) { y -= 2; montee--; } // Il monte : y diminue de 2 (« -= » retire).
                                                // 16 × 2 = 32 pixels de haut.
        else if (y < SOL) y += 2;              // Sinon, s'il est en l'air : il redescend.

        sprite(0, 76, y, HEROS); // On repose le héros.
        image();                  // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

A fait sauter, et le personnage retombe au sol.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ il part du sol
- ✓ A le fait monter (120 → 100)
- ✓ il retombe exactement au sol ($y = 120$)
- ✓ tenir A ne le fait pas coller au plafond

Ensuite : **52. defiler(), et l'octet qui boucle** — chapitre 7 : Le mouvement. · `gameboy3` — écrire une cartouche Game Boy en C++